Verilog HDL:
Stratified Event Queues

Pravin Zode

Outline

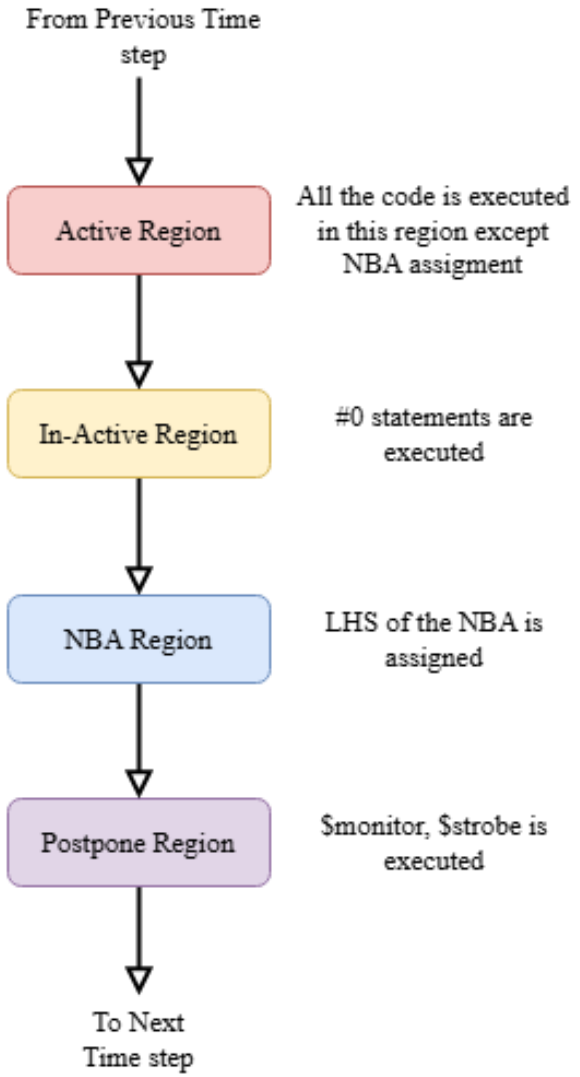
- Verilog Event Regions
  - Active region
  - In-active region
  - NBA or Non-Blocking assignment region
  - Postponed region
- Examples
- Summary

Verilog Event Regions

Verilog is a time-dependent simulator

In Verilog, each time step is divided into 4 different regions in which various statements are executed to simulate the hardware accurately

- Active region
- In-active region
- NBA or Non-Blocking assignment region
- Postponed region



These regions ensure **deterministic simulation** and **correct timing behavior**

Verilog Event Regions

Active Region

- Executes blocking assignments (=)
- Evaluates continuous assignments (assign)
- Executes procedural statements (always/initial blocks)
- Order of execution is not guaranteed (can lead to race conditions)

Inactive Region

- Holds events scheduled with #0 delay
- Executes after the Active region completes
- Helps in resolving race conditions
- No new evaluations, only delayed events processed

Verilog Event Regions

NBA (Non-Blocking Assignment)

- Executes non-blocking assignments (<=)
- RHS evaluated earlier, LHS updated in this region
- Ensures parallel behavior (like real hardware registers)
- Avoids race conditions in sequential logic

Postponed Region

- Used for monitoring and observation
- Executes $monitor, $strobe, waveform dumping
- No effect on design values (read-only phase)
- Final stable values are observed here

Active Region (Blocking Assignment Issue)

Concept

- Blocking assignments execute immediately
- Order is not guaranteed → race condition

```
1   module active_region_example;
2     reg a, b;
3
4     initial begin
5       a = 0;
6       b = 1;
7
8       #5 a = b;   // executes in active region
9          b = a;   // depends on updated 'a'
10
11      #5 $display("a = %0d, b = %0d", a, b);
12    end
13  endmodule
```

Observation

- Output depends on execution order
- Can produce unexpected results

Output:

```
a = 1, b = 1
```

Inactive Region (#0 Delay)

Concept

- #0 delays move execution to inactive region
- Helps separate operations within same time step

```
1   module inactive_region_example;
2     reg a, b;
3
4     initial begin
5       a = 0;
6       b = 1;
7
8       #5 a = b;       // Active region
9          #0 b = a;    // Inactive region
10
11      #5 $display("a = %0d, b = %0d", a, b);
12    end
13  endmodule
```

Observation

- b = a executes after active region completes
- Helps avoid race condition

Output:

a = 1, b = 1

NBA Region (Non-Blocking Assignment)

Concept

- RHS evaluated first
- LHS updated later in NBA region
- Mimics parallel hardware behavior

```
1  module nba_region_example;
2    reg a, b;
3
4    initial begin
5      a = 0;
6      b = 1;
7
8      #5 a <= b;   // scheduled in NBA
9         b <= a;   // uses old value of 'a'
10
11     #5 $display("a = %0d, b = %0d", a, b);
12   end
13 endmodule
```

Observation

- Both assignments happen simultaneously
- Output: values swap correctly
- No race condition

Output:

```
a = 1, b = 0
```

Postponed Region ($monitor / $strobe)

Concept

- Executes after all updates
- Used for final stable value observation

```
1  module postponed_region_example;
2    reg a, b;
3
4    initial begin
5      a = 0;
6      b = 1;
7
8      $monitor("Time=%0t a=%0d b=%0d", $time, a, b);
9
10     #5 a <= b;
11        b <= a;
12
13     #5 $strobe("STROBE: a=%0d b=%0d", a, b);
14   end
15 endmodule
```

Observation

- $monitor → tracks changes continuously
- $strobe → prints final updated values
- Executed in postponed region

Output:

```
Time=0 a=0 b=1
Time=5 a=1 b=0
STROBE: a=1 b=0
```

Summary

- Blocking (=) → Active region → sequential, risky
- #0 delay → Inactive region → ordering control
- Non-blocking (<=) → NBA → parallel, safe
- $monitor/$strobe → Postponed → observation only



Thank you !

Happy Learning